# Level 4: JOINs

*JOINs combine data spread across tables: INNER (only matches), LEFT/RIGHT (keep all of one side), FULL OUTER (keep everything), SELF (a table joined to itself), and CROSS (every combination). This is the biggest level in the book -- by the end you'll recognize the 'junction table' pattern (orders/order_items, students/enrollments/courses) everywhere.*

## Q76. INNER JOIN employee & department MEDIUM

```
SELECT e.name, e.department, d.location, d.budget
FROM employees e
INNER JOIN departments d ON e.department = d.dept_name;
```

**WHAT THIS QUERY DOES**

A `JOIN` combines rows from two tables based on a matching condition — here, an employee's `department` text must match a department's `dept_name`. `INNER JOIN` (the default kind, often just written `JOIN`) only keeps rows where a match is found on *both* sides — an employee whose department didn't exist in the `departments` table would simply be left out entirely, and a department with no employees wouldn't appear either. Aliasing the tables as `e` and `d` is a convention that keeps column references short once you're juggling two or more tables.

**RESULT ON OUR SAMPLE DATA**

| name | department | location | budget |
|---|---|---|---|
| Aarav Sharma | IT | Mumbai | 5,000,000 |
| Priya Patel | HR | Delhi | 2,000,000 |
| Rahul Singh | Finance | Mumbai | 3,000,000 |
| Neha Gupta | IT | Mumbai | 5,000,000 |
| Vikram Reddy | IT | Mumbai | 5,000,000 |
| Ananya Menon | HR | Delhi | 2,000,000 |
| Karthik Nair | IT | Mumbai | 5,000,000 |
| Meera Krishnan | Marketing | Bangalore | 2,500,000 |
| Arjun Reddy | IT | Mumbai | 5,000,000 |
| Sneha Patel | Marketing | Bangalore | 2,500,000 |
| Rajesh Kumar | Finance | Mumbai | 3,000,000 |
| Deepa Iyer | HR | Delhi | 2,000,000 |
| Suresh Babu | IT | Mumbai | 5,000,000 |
| Kavya Nair | Finance | Mumbai | 3,000,000 |
| Ravi Teja | IT | Mumbai | 5,000,000 |

15 rows returned.

## Q77. LEFT JOIN MEDIUM

```
SELECT e.name, e.department, d.location
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name;
```

**WHAT THIS QUERY DOES**

LEFT JOIN keeps *every* row from the left table (employees) no matter what — if a matching department isn't found, the department columns simply come back as NULL instead of the row being dropped. This is the key difference from INNER JOIN: INNER JOIN can silently lose rows that don't match; LEFT JOIN guarantees the 'main' table's rows all survive, which is exactly why it's the go-to choice whenever you specifically want to find *unmatched* rows (see Q81).

**RESULT ON OUR SAMPLE DATA**

| name | department | location |
|---|---|---|
| Aarav Sharma | IT | Mumbai |
| Priya Patel | HR | Delhi |
| Rahul Singh | Finance | Mumbai |
| Neha Gupta | IT | Mumbai |
| Vikram Reddy | IT | Mumbai |
| Ananya Menon | HR | Delhi |
| Karthik Nair | IT | Mumbai |
| Meera Krishnan | Marketing | Bangalore |
| Arjun Reddy | IT | Mumbai |
| Sneha Patel | Marketing | Bangalore |
| Rajesh Kumar | Finance | Mumbai |
| Deepa Iyer | HR | Delhi |
| Suresh Babu | IT | Mumbai |
| Kavya Nair | Finance | Mumbai |
| Ravi Teja | IT | Mumbai |

15 rows returned.

## Q78. RIGHT JOIN MEDIUM

```
SELECT e.name, d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d ON e.department = d.dept_name;
```

**WHAT THIS QUERY DOES**

The mirror image of LEFT JOIN: now every row from the *right* table (departments) is guaranteed to survive, with employee columns coming back NULL if no employee matches. Notice the row count here is 16, one more than employees alone (15) — that's the 'Operations' department showing up with a NULL employee name, since no one in our sample data works there. RIGHT JOIN is used far less often than LEFT JOIN in practice, mostly because you can always rewrite a RIGHT JOIN as a LEFT JOIN by swapping which table you list first.

**RESULT ON OUR SAMPLE DATA**

| name | dept_name | location |
|---|---|---|
| Aarav Sharma | IT | Mumbai |
| Priya Patel | HR | Delhi |
| Rahul Singh | Finance | Mumbai |
| Neha Gupta | IT | Mumbai |
| Vikram Reddy | IT | Mumbai |
| Ananya Menon | HR | Delhi |
| Karthik Nair | IT | Mumbai |
| Meera Krishnan | Marketing | Bangalore |
| Arjun Reddy | IT | Mumbai |
| Sneha Patel | Marketing | Bangalore |
| Rajesh Kumar | Finance | Mumbai |
| Deepa Iyer | HR | Delhi |
| Suresh Babu | IT | Mumbai |
| Kavya Nair | Finance | Mumbai |
| Ravi Teja | IT | Mumbai |
| *NULL* | Operations | Pune |

16 rows returned.

## Q79. FULL OUTER JOIN (MySQL workaround) HARD

```
SELECT e.name, d.dept_name, d.location
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name
```

```
UNION
SELECT e.name, d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d ON e.department = d.dept_name;
```

**WHAT THIS QUERY DOES**

A `FULL OUTER JOIN` keeps every row from *both* tables, matched where possible and NULL-padded where not. Older MySQL versions don't support `FULL OUTER JOIN` directly, so this classic workaround unions together a LEFT JOIN (catches unmatched employees) with a RIGHT JOIN (catches unmatched departments) — and since `UNION` automatically removes duplicate rows, the rows that matched on both sides don't get counted twice. Modern SQLite (and modern MySQL/Postgres) support `FULL OUTER JOIN` natively, so the live example below runs the simpler direct syntax and produces the same result.

**RESULT ON OUR SAMPLE DATA**

| name | dept_name | location |
|---|---|---|
| Aarav Sharma | IT | Mumbai |
| Priya Patel | HR | Delhi |
| Rahul Singh | Finance | Mumbai |
| Neha Gupta | IT | Mumbai |
| Vikram Reddy | IT | Mumbai |
| Ananya Menon | HR | Delhi |
| Karthik Nair | IT | Mumbai |
| Meera Krishnan | Marketing | Bangalore |
| Arjun Reddy | IT | Mumbai |
| Sneha Patel | Marketing | Bangalore |
| Rajesh Kumar | Finance | Mumbai |
| Deepa Iyer | HR | Delhi |
| Suresh Babu | IT | Mumbai |
| Kavya Nair | Finance | Mumbai |
| Ravi Teja | IT | Mumbai |
| *NULL* | Operations | Pune |

16 rows returned.

## Q80. SELF JOIN employee-manager MEDIUM

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

**WHAT THIS QUERY DOES**

A **self join** joins a table to itself — here, `employees` is used twice, once aliased as `e` (playing the role of 'the employee') and once as `m` (playing the role of 'their manager'). This is exactly how you unravel a self-referencing column like `manager_id`, which stores another row's ID from the very same table, into a readable name-to-name relationship. `LEFT JOIN` is used specifically so the one employee with no manager (`manager_id IS NULL`) still appears, just with a NULL manager name, rather than being dropped.

**RESULT ON OUR SAMPLE DATA**

| employee | manager |
|---|---|
| Aarav Sharma | *NULL* |
| Priya Patel | Aarav Sharma |
| Rahul Singh | Aarav Sharma |
| Neha Gupta | Aarav Sharma |
| Vikram Reddy | Aarav Sharma |
| Ananya Menon | Priya Patel |
| Karthik Nair | Aarav Sharma |
| Meera Krishnan | Vikram Reddy |
| Arjun Reddy | Vikram Reddy |
| Sneha Patel | Meera Krishnan |
| Rajesh Kumar | Rahul Singh |
| Deepa Iyer | Priya Patel |
| Suresh Babu | Vikram Reddy |
| Kavya Nair | Rahul Singh |
| Ravi Teja | Aarav Sharma |

15 rows returned.

## Q81. Employees without department MEDIUM

```
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name
WHERE d.dept_id IS NULL;
```

**WHAT THIS QUERY DOES**

This is the classic 'find unmatched rows' pattern: LEFT JOIN keeps every employee, filling in NULL for any department columns that didn't match — and then `WHERE d.dept_id IS NULL` keeps only the rows where that NULL-filling actually happened, i.e. employees whose department didn't exist in the departments table. In our sample data every employee's department is valid, so this correctly returns 0 rows

— but this exact pattern is one of the most useful JOIN techniques for spotting broken/orphaned foreign-key references in real data.

**RESULT ON OUR SAMPLE DATA**

| name |
|---|
| *(no rows returned)* |

0 rows returned.

## Q82. Departments without employees MEDIUM

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_name = e.department
WHERE e.emp_id IS NULL;
```

**WHAT THIS QUERY DOES**

The mirror image of Q81 — LEFT JOIN starting from `departments` this time, so every department survives, and the NULL-check then reveals which departments have no employees at all. 'Operations' is the one department in our sample data that exists but has nobody assigned to it yet.

**RESULT ON OUR SAMPLE DATA**

| dept_name |
|---|
| Operations |

1 row returned.

## Q83. Orders with customers EASY

```
SELECT o.order_id, c.name, o.order_date, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

**WHAT THIS QUERY DOES**

A plain INNER JOIN attaching each order to the customer who placed it — since `orders` only stores a `customer_id` number, this is what lets you show the actual customer *name* alongside each order instead of a meaningless ID.

**RESULT ON OUR SAMPLE DATA**

| order_id | name | order_date | amount |
|---|---|---|---|
| 101 | TechCorp | 2024-01-10 | 15,000 |
| 102 | TechCorp | 2024-01-15 | 23,000 |

| order_id | name | order_date | amount |
|---|---|---|---|
| 103 | GlobalSolutions | 2024-01-20 | 12,000 |
| 104 | InnovationLabs | 2024-02-01 | 50,000 |
| 105 | InnovationLabs | 2024-02-05 | 18,000 |
| 106 | AlphaInc | 2024-02-10 | 32,000 |
| 107 | BetaSystems | 2024-02-15 | 45,000 |
| 108 | GammaTech | 2024-03-01 | 21,000 |
| 109 | DeltaCorp | 2024-03-05 | 67,000 |
| 110 | EpsilonSoft | 2024-03-10 | 34,000 |
| 111 | ThetaSolutions | 2024-03-15 | 28,000 |
| 112 | OmegaLabs | 2024-04-01 | 42,000 |
| 113 | TechCorp | 2024-04-05 | 15,000 |
| 114 | GlobalSolutions | 2024-04-10 | 33,000 |
| 115 | InnovationLabs | 2024-04-15 | 29,000 |

15 rows returned.

## Q84. Customers without orders MEDIUM

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

**WHAT THIS QUERY DOES**

Same 'LEFT JOIN then check for NULL' technique as Q81/Q82, now finding customers who exist in the `customers` table but have never placed an order. Every customer in our sample data happens to have at least one order, so this correctly returns 0 rows here.

**RESULT ON OUR SAMPLE DATA**

| name |
|---|
| *(no rows returned)* |

0 rows returned.

## Q85. Products never ordered MEDIUM

```
SELECT p.product_name
FROM products p
```

```
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
```

**WHAT THIS QUERY DOES**

The same unmatched-rows pattern again, this time surfacing products that have never appeared in any order — useful for spotting slow-moving or completely dead inventory.

**RESULT ON OUR SAMPLE DATA**

| product_name |
|---|
| Webcam HD |
| Printer |
| USB Hub |

3 rows returned.

## Q86. Orders with product details MEDIUM

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

**WHAT THIS QUERY DOES**

Your first **three-table join**: `orders` connects to `order_items` (to find out which products and quantities were in each order), which in turn connects to `products` (to get readable product names instead of just IDs). Chaining JOINs like this — each one attaching one more related table — is exactly how real reporting queries assemble a complete picture spread across a normalized database.

**RESULT ON OUR SAMPLE DATA**

| order_id | product_name | quantity | price |
|---|---|---|---|
| 101 | Laptop Pro | 2 | 75,000 |
| 101 | Wireless Mouse | 1 | 1,500 |
| 102 | Enterprise Software | 1 | 100,000 |
| 103 | 27" Monitor | 1 | 25,000 |
| 104 | Laptop Pro | 1 | 75,000 |
| 104 | External SSD | 2 | 12,000 |
| 105 | Enterprise Software | 1 | 100,000 |
| 106 | 27" Monitor | 2 | 25,000 |
| 107 | Laptop Pro | 3 | 75,000 |
| 108 | Cloud Storage Plan | 1 | 50,000 |

| order_id | product_name | quantity | price |
|---|---|---|---|
| 109 | Laptop Pro | 2 | 75,000 |
| 110 | External SSD | 2 | 12,000 |
| 111 | Enterprise Software | 1 | 100,000 |
| 112 | Laptop Pro | 1 | 75,000 |
| 113 | Mechanical Keyboard | 1 | 5,000 |

15 rows returned.

## Q87. Students with courses (Example structure) MEDIUM

```
-- Assuming tables exist: students, courses, enrollments
SELECT s.name, c.course_name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

**WHAT THIS QUERY DOES**

*This query uses a hypothetical schema the cheat sheet sketches as an example, separate from this book's main company database — so a small illustrative `students` / `courses` / `enrollments` dataset (4 students, 4 courses) has been added just to demonstrate it concretely.* The structure is a classic **many-to-many relationship**: one student can take many courses, and one course can have many students, so a middle 'junction' table (`enrollments`) is needed to connect them — you can't link students directly to courses with a single foreign key the way employees link directly to one department.

**RESULT ON OUR SAMPLE DATA**

| name | course_name |
|---|---|
| Aditi Rao | Data Structures |
| Aditi Rao | Operating Systems |
| Bilal Khan | Data Structures |
| Chen Wei | Computer Networks |
| Diana Cruz | Data Structures |
| Diana Cruz | Computer Networks |

6 rows returned.

## Q88. Courses without students MEDIUM

```
-- Assuming tables exist
SELECT c.course_name
```

```
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;
```

**WHAT THIS QUERY DOES**

Same unmatched-rows LEFT JOIN pattern as Q81/84/85, applied to the illustrative students/courses schema — 'Compiler Design' was deliberately left with zero enrollments in the sample data so this query has a genuine, non-empty result to show.

**RESULT ON OUR SAMPLE DATA**

| course_name |
|---|
| Compiler Design |

1 row returned.

## Q89. Employees with projects MEDIUM

```
SELECT e.name, p.project_name
FROM employees e
JOIN projects p ON e.department = p.department;
```

**WHAT THIS QUERY DOES**

Joining on `department` (rather than a dedicated project-assignment table) means this connects *every* employee to *every* project run by their department — not just one specific project they're personally assigned to. That's why the result has 29 rows even though there are only 15 employees: IT alone runs 3 active projects, so every IT employee shows up 3 times, once per project. This is worth noticing as a caution about JOINs: joining on a shared category column (department) rather than a true one-to-one assignment key can multiply rows in ways that aren't always what you want.

**RESULT ON OUR SAMPLE DATA**

| name | project_name |
|---|---|
| Aarav Sharma | AI Implementation |
| Aarav Sharma | Cloud Migration |
| Aarav Sharma | Employee Portal |
| Priya Patel | Talent Acquisition |
| Rahul Singh | Financial Audit |
| Neha Gupta | AI Implementation |
| Neha Gupta | Cloud Migration |
| Neha Gupta | Employee Portal |
| Vikram Reddy | AI Implementation |

| name | project_name |
|---|---|
| Vikram Reddy | Cloud Migration |
| Vikram Reddy | Employee Portal |
| Ananya Menon | Talent Acquisition |
| Karthik Nair | AI Implementation |
| Karthik Nair | Cloud Migration |
| Karthik Nair | Employee Portal |
| Meera Krishnan | Market Research |
| Arjun Reddy | AI Implementation |
| Arjun Reddy | Cloud Migration |
| Arjun Reddy | Employee Portal |
| Sneha Patel | Market Research |

Showing first 20 of 29 rows.

## Q90. Projects without employees MEDIUM

```
SELECT p.project_name
FROM projects p
LEFT JOIN employees e ON p.department = e.department
WHERE e.emp_id IS NULL;
```

**WHAT THIS QUERY DOES**

Same unmatched-rows technique again — finds any project belonging to a department that currently has no employees at all. Since every department that has a project also has staff in our sample data, this returns 0 rows.

**RESULT ON OUR SAMPLE DATA**

| project_name |
|---|
| *(no rows returned)* |

0 rows returned.

## Q91. Three-table join - Employee, Department, Projects HARD

```
SELECT e.name, e.department, d.location, p.project_name
FROM employees e
JOIN departments d ON e.department = d.dept_name
LEFT JOIN projects p ON e.department = p.department;
```

**WHAT THIS QUERY DOES**

Mixing join types in one query: an `INNER JOIN` to `departments` (every employee's department is expected to exist, so no need to preserve unmatched rows there) combined with a `LEFT JOIN` to `projects` (so employees in departments with zero projects still show up, with a NULL project name, rather than disappearing). Choosing INNER vs LEFT independently for each join in a multi-join query, based on whether you want to risk losing unmatched rows, is an important practical skill.

**RESULT ON OUR SAMPLE DATA**

| name | department | location | project_name |
|---|---|---|---|
| Aarav Sharma | IT | Mumbai | AI Implementation |
| Aarav Sharma | IT | Mumbai | Cloud Migration |
| Aarav Sharma | IT | Mumbai | Employee Portal |
| Priya Patel | HR | Delhi | Talent Acquisition |
| Rahul Singh | Finance | Mumbai | Financial Audit |
| Neha Gupta | IT | Mumbai | AI Implementation |
| Neha Gupta | IT | Mumbai | Cloud Migration |
| Neha Gupta | IT | Mumbai | Employee Portal |
| Vikram Reddy | IT | Mumbai | AI Implementation |
| Vikram Reddy | IT | Mumbai | Cloud Migration |
| Vikram Reddy | IT | Mumbai | Employee Portal |
| Ananya Menon | HR | Delhi | Talent Acquisition |
| Karthik Nair | IT | Mumbai | AI Implementation |
| Karthik Nair | IT | Mumbai | Cloud Migration |
| Karthik Nair | IT | Mumbai | Employee Portal |
| Meera Krishnan | Marketing | Bangalore | Market Research |
| Arjun Reddy | IT | Mumbai | AI Implementation |
| Arjun Reddy | IT | Mumbai | Cloud Migration |
| Arjun Reddy | IT | Mumbai | Employee Portal |
| Sneha Patel | Marketing | Bangalore | Market Research |

Showing first 20 of 29 rows.

## Q92. Four-table join - Customer, Orders, OrderItems, Products

HARD

```
SELECT c.name AS customer, o.order_id, p.product_name, oi.quantity, oi.price
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

**WHAT THIS QUERY DOES**

Extending the three-table join from Q86 by one more table: customer → their orders → each order's line items → each item's product details. Four tables chained together like this is a very normal size for a real invoice/receipt-style report — each JOIN just walks one more step along the natural relationships between the tables.

**RESULT ON OUR SAMPLE DATA**

| customer | order_id | product_name | quantity | price |
|---|---|---|---|---|
| TechCorp | 101 | Laptop Pro | 2 | 75,000 |
| TechCorp | 101 | Wireless Mouse | 1 | 1,500 |
| TechCorp | 102 | Enterprise Software | 1 | 100,000 |
| GlobalSolutions | 103 | 27" Monitor | 1 | 25,000 |
| InnovationLabs | 104 | Laptop Pro | 1 | 75,000 |
| InnovationLabs | 104 | External SSD | 2 | 12,000 |
| InnovationLabs | 105 | Enterprise Software | 1 | 100,000 |
| AlphaInc | 106 | 27" Monitor | 2 | 25,000 |
| BetaSystems | 107 | Laptop Pro | 3 | 75,000 |
| GammaTech | 108 | Cloud Storage Plan | 1 | 50,000 |
| DeltaCorp | 109 | Laptop Pro | 2 | 75,000 |
| EpsilonSoft | 110 | External SSD | 2 | 12,000 |
| ThetaSolutions | 111 | Enterprise Software | 1 | 100,000 |
| OmegaLabs | 112 | Laptop Pro | 1 | 75,000 |
| TechCorp | 113 | Mechanical Keyboard | 1 | 5,000 |

15 rows returned.

## Q93. Employee → Department → Location EASY

```
SELECT e.name, e.department, d.location, d.budget
FROM employees e
JOIN departments d ON e.department = d.dept_name;
```

**WHAT THIS QUERY DOES**

Identical to Q76 — the cheat sheet repeats this exact join here to reinforce that 'follow a foreign key to pull in details from the related table' is the single most common JOIN use case you'll write, over and over, in real applications.

**RESULT ON OUR SAMPLE DATA**

| name | department | location | budget |
|---|---|---|---|
| Aarav Sharma | IT | Mumbai | 5,000,000 |
| Priya Patel | HR | Delhi | 2,000,000 |

| name | department | location | budget |
|---|---|---|---|
| Rahul Singh | Finance | Mumbai | 3,000,000 |
| Neha Gupta | IT | Mumbai | 5,000,000 |
| Vikram Reddy | IT | Mumbai | 5,000,000 |
| Ananya Menon | HR | Delhi | 2,000,000 |
| Karthik Nair | IT | Mumbai | 5,000,000 |
| Meera Krishnan | Marketing | Bangalore | 2,500,000 |
| Arjun Reddy | IT | Mumbai | 5,000,000 |
| Sneha Patel | Marketing | Bangalore | 2,500,000 |
| Rajesh Kumar | Finance | Mumbai | 3,000,000 |
| Deepa Iyer | HR | Delhi | 2,000,000 |
| Suresh Babu | IT | Mumbai | 5,000,000 |
| Kavya Nair | Finance | Mumbai | 3,000,000 |
| Ravi Teja | IT | Mumbai | 5,000,000 |

15 rows returned.

## Q94. Customer → Order → Product MEDIUM

```
SELECT c.name, o.order_date, p.product_name, oi.quantity
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

**WHAT THIS QUERY DOES**

Essentially the same four-table chain as Q92, just selecting a slightly different set of output columns (order date instead of order ID and price) — a reminder that the JOIN structure of a query and the specific columns you choose to display are two independent decisions.

**RESULT ON OUR SAMPLE DATA**

| name | order_date | product_name | quantity |
|---|---|---|---|
| TechCorp | 2024-01-10 | Laptop Pro | 2 |
| TechCorp | 2024-01-10 | Wireless Mouse | 1 |
| TechCorp | 2024-01-15 | Enterprise Software | 1 |
| GlobalSolutions | 2024-01-20 | 27" Monitor | 1 |
| InnovationLabs | 2024-02-01 | Laptop Pro | 1 |
| InnovationLabs | 2024-02-01 | External SSD | 2 |
| InnovationLabs | 2024-02-05 | Enterprise Software | 1 |
| AlphaInc | 2024-02-10 | 27" Monitor | 2 |

| name | order_date | product_name | quantity |
|---|---|---|---|
| BetaSystems | 2024-02-15 | Laptop Pro | 3 |
| GammaTech | 2024-03-01 | Cloud Storage Plan | 1 |
| DeltaCorp | 2024-03-05 | Laptop Pro | 2 |
| EpsilonSoft | 2024-03-10 | External SSD | 2 |
| ThetaSolutions | 2024-03-15 | Enterprise Software | 1 |
| OmegaLabs | 2024-04-01 | Laptop Pro | 1 |
| TechCorp | 2024-04-05 | Mechanical Keyboard | 1 |

15 rows returned.

## Q95. Library joins (Example) MEDIUM

```
-- Members, Books, Loans
SELECT m.name, b.title, l.loan_date
FROM members m
JOIN loans l ON m.member_id = l.member_id
JOIN books b ON l.book_id = b.book_id;
```

**WHAT THIS QUERY DOES**

*Another illustrative hypothetical schema — a small library dataset (3 members, 3 books, 4 loans) was added to demonstrate this concretely.* Structurally, this is the same three-table chain as Q86 (orders → order_items → products): a middle table (`loans`) records *which* member borrowed *which* book and when, connecting two tables that have no direct relationship to each other.

**RESULT ON OUR SAMPLE DATA**

| name | title | loan_date |
|---|---|---|
| Aarav Sharma | Clean Code | 2024-05-01 |
| Aarav Sharma | Design Patterns | 2024-05-10 |
| Priya Patel | The Pragmatic Programmer | 2024-05-15 |
| Rahul Singh | Clean Code | 2024-06-01 |

4 rows returned.

## Q96. Hospital joins (Example) MEDIUM

```
-- Patients, Doctors, Appointments
SELECT p.name, d.name, a.appointment_date
FROM patients p
```

```
JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id;
```

**WHAT THIS QUERY DOES**

*Illustrative hospital schema (3 patients, 2 doctors, 4 appointments) added for a concrete demonstration.* The `appointments` table plays the same 'junction table' role as `loans` did in Q95 or `order_items` in Q86 — it's what links two otherwise-unrelated tables (patients, doctors) together, recording who saw whom and when.

**RESULT ON OUR SAMPLE DATA**

| name | name | appointment_date |
|---|---|---|
| Meera Krishnan | Dr. Kapoor | 2024-03-01 |
| Arjun Reddy | Dr. Iyer | 2024-03-03 |
| Sneha Patel | Dr. Kapoor | 2024-03-05 |
| Meera Krishnan | Dr. Iyer | 2024-04-02 |

4 rows returned.

## Q97. Banking joins (Example) MEDIUM

```
-- Customers, Accounts, Transactions
SELECT c.name, a.account_number, t.amount, t.transaction_date
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
JOIN transactions t ON a.account_id = t.account_id;
```

**WHAT THIS QUERY DOES**

*Illustrative banking schema added (2 bank customers, 2 accounts, 3 transactions) — kept in separate tables named `bank_customers/accounts/transactions` so they don't collide with this book's main `customers` table.* Note this is a slightly deeper chain than a simple two-hop join: customer → their account(s) → that account's transactions, a very standard shape for financial data where a customer's money movements are always tied to a specific account, not to the customer directly.

**RESULT ON OUR SAMPLE DATA**

| name | account_number | amount | transaction_date |
|---|---|---|---|
| TechCorp | AC1001 | 5,000 | 2024-01-05 |
| TechCorp | AC1001 | -1,200 | 2024-01-10 |
| GlobalSolutions | AC1002 | 8,000 | 2024-01-12 |

3 rows returned.

## Q98. Airline joins (Example) MEDIUM

```
-- Passengers, Flights, Bookings
SELECT p.name, f.flight_number, b.booking_date
FROM passengers p
JOIN bookings b ON p.passenger_id = b.passenger_id
JOIN flights f ON b.flight_id = f.flight_id;
```

**WHAT THIS QUERY DOES**

*Illustrative airline schema added (2 passengers, 2 flights, 2 bookings).* Same junction-table shape once more — `bookings` connects passengers to flights, since one passenger can book many flights and one flight carries many passengers (a many-to-many relationship, just like Q87's students/courses).

**RESULT ON OUR SAMPLE DATA**

| name | flight_number | booking_date |
|---|---|---|
| Karthik Nair | AI-202 | 2024-02-01 |
| Kavya Nair | 6E-501 | 2024-02-03 |

2 rows returned.

## Q99. Hotel joins (Example) MEDIUM

```
-- Guests, Rooms, Reservations
SELECT g.name, r.room_number, res.check_in_date
FROM guests g
JOIN reservations res ON g.guest_id = res.guest_id
JOIN rooms r ON res.room_id = r.room_id;
```

**WHAT THIS QUERY DOES**

*Illustrative hotel schema added (2 guests, 2 rooms, 2 reservations).* By now the pattern should feel very familiar: whenever two real-world entities (guests, rooms) can each relate to many of the other, a junction table (`reservations`) in the middle is what SQL uses to record and query those specific pairings.

**RESULT ON OUR SAMPLE DATA**

| name | room_number | check_in_date |
|---|---|---|
| Suresh Babu | 101 | 2024-03-10 |
| Ravi Teja | 102 | 2024-03-12 |

2 rows returned.

## Q100. Movie booking joins (Example) HARD

```
-- Users, Movies, Showtimes, Bookings
SELECT u.name, m.title, s.show_time, b.booking_date
FROM users u
JOIN bookings b ON u.user_id = b.user_id
JOIN showtimes s ON b.showtime_id = s.showtime_id
JOIN movies m ON s.movie_id = m.movie_id;
```

**WHAT THIS QUERY DOES**

*Illustrative movie-booking schema added (2 users, 2 movies, 2 showtimes, 2 bookings; tables named `app_users` and `movie_bookings` in the live example to avoid clashing with SQLite's reserved words and this book's other tables).* This is a four-table chain with *two* junction-style hops: a movie has many showtimes, and a showtime has many bookings from different users — so reaching from 'user' to 'movie title' takes two intermediate steps (showtimes, then bookings) rather than one.

**RESULT ON OUR SAMPLE DATA**

| name | title | show_time | booking_date |
|---|---|---|---|
| Neha Gupta | Interstellar Redux | 2024-04-01 18:00 | 2024-03-28 |
| Rajesh Kumar | The Last Algorithm | 2024-04-01 21:00 | 2024-03-29 |

2 rows returned.

## Q101. Employees and their project count MEDIUM

```
SELECT e.name, COUNT(p.project_id) AS project_count
FROM employees e
LEFT JOIN projects p ON e.department = p.department
GROUP BY e.emp_id, e.name;
```

**WHAT THIS QUERY DOES**

Combining JOIN with GROUP BY and COUNT, all in one query: LEFT JOIN attaches every project run by an employee's department (so employees in project-less departments still appear, with a count of 0 rather than vanishing), and grouping by employee then counts how many project rows each one picked up. Grouping by `e.emp_id` as well as `e.name` avoids accidentally merging two different employees who happen to share the same name.

**RESULT ON OUR SAMPLE DATA**

| name | project_count |
|---|---|
| Aarav Sharma | 3 |
| Priya Patel | 1 |

| name | project_count |
|---|---|
| Rahul Singh | 1 |
| Neha Gupta | 3 |
| Vikram Reddy | 3 |
| Ananya Menon | 1 |
| Karthik Nair | 3 |
| Meera Krishnan | 1 |
| Arjun Reddy | 3 |
| Sneha Patel | 1 |
| Rajesh Kumar | 1 |
| Deepa Iyer | 1 |
| Suresh Babu | 3 |
| Kavya Nair | 1 |
| Ravi Teja | 3 |

15 rows returned.

## Q102. Departments with total project budget MEDIUM

```
SELECT d.dept_name, SUM(p.budget) AS total_budget
FROM departments d
LEFT JOIN projects p ON d.dept_name = p.department
GROUP BY d.dept_id, d.dept_name;
```

**WHAT THIS QUERY DOES**

Same JOIN + GROUP BY + aggregate combination as Q101, now summing project budgets per department instead of counting rows per employee. LEFT JOIN ensures 'Operations' (which has no projects) still shows up in the result, with a NULL total budget rather than being silently dropped.

**RESULT ON OUR SAMPLE DATA**

| dept_name | total_budget |
|---|---|
| Finance | 500,000 |
| HR | 400,000 |
| IT | 4,300,000 |
| Marketing | 300,000 |
| Operations | *NULL* |

5 rows returned.

## Q103. Customers with highest order amount MEDIUM

```
SELECT c.name, MAX(o.amount) AS max_order
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY max_order DESC LIMIT 3;
```

**WHAT THIS QUERY DOES**

JOIN to attach each customer's orders, GROUP BY to find each customer's single biggest order, then the familiar 'sort descending, take top N' pattern (Q14) to surface the 3 customers whose largest single order was the biggest of all.

**RESULT ON OUR SAMPLE DATA**

| name | max_order |
|---|---|
| DeltaCorp | 67,000 |
| InnovationLabs | 50,000 |
| BetaSystems | 45,000 |

3 rows returned.

## Q104. Products with total quantity sold MEDIUM

```
SELECT p.product_name, SUM(oi.quantity) AS total_sold
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_sold DESC;
```

**WHAT THIS QUERY DOES**

LEFT JOIN here matters for a subtle reason: it guarantees products that have *never* been ordered still appear in the result (with a total of NULL/0), rather than silently disappearing the way an INNER JOIN would make them — useful for a true 'sales by product' report where you want to see the zero-sellers too, not just a list of what happened to sell.

**RESULT ON OUR SAMPLE DATA**

| product_name | total_sold |
|---|---|
| Laptop Pro | 9 |
| External SSD | 4 |
| 27" Monitor | 3 |
| Enterprise Software | 3 |
| Wireless Mouse | 1 |

| product_name | total_sold |
|---|---|
| Mechanical Keyboard | 1 |
| Cloud Storage Plan | 1 |
| Webcam HD | *NULL* |
| Printer | *NULL* |
| USB Hub | *NULL* |

10 rows returned.

## Q105. Employee salary vs department average HARD

```
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN (SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY
department) d
ON e.department = d.department;
```

**WHAT THIS QUERY DOES**

This introduces joining against a **subquery acting as a temporary table** (sometimes called a derived table): the parenthesized `SELECT ... GROUP BY department` first computes each department's average salary as its own mini result set, which is then joined back to `employees` just like any real table would be. This is a very common technique whenever you need to compare each individual row against an aggregate computed from its own group — and is a clean, JOIN-based alternative to the correlated subquery approach used back in Q62/Q123.

**RESULT ON OUR SAMPLE DATA**

| name | salary | avg_salary |
|---|---|---|
| Aarav Sharma | 75,000 | 84,714.29 |
| Priya Patel | 65,000 | 63,000 |
| Rahul Singh | 85,000 | 74,000 |
| Neha Gupta | 72,000 | 84,714.29 |
| Vikram Reddy | 90,000 | 84,714.29 |
| Ananya Menon | 60,000 | 63,000 |
| Karthik Nair | 95,000 | 84,714.29 |
| Meera Krishnan | 68,000 | 63,000 |
| Arjun Reddy | 82,000 | 84,714.29 |
| Sneha Patel | 58,000 | 63,000 |
| Rajesh Kumar | 70,000 | 74,000 |
| Deepa Iyer | 64,000 | 63,000 |
| Suresh Babu | 88,000 | 84,714.29 |

| name | salary | avg_salary |
|---|---|---|
| Kavya Nair | 67,000 | 74,000 |
| Ravi Teja | 91,000 | 84,714.29 |

15 rows returned.

## Q106. Multiple join with conditions HARD

```
SELECT c.name, o.order_date, SUM(oi.quantity * oi.price) AS total
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status = 'Completed'
AND o.order_date > '2024-01-01'
GROUP BY c.customer_id, c.name, o.order_date;
```

**WHAT THIS QUERY DOES**

A realistic 'everything at once' query: three tables joined together, filtered down with WHERE (only completed orders after a certain date), and grouped to compute each qualifying order's true total value (`quantity × price`, summed across every line item in that order). This is close to what an actual sales report query looks like in production — JOINs, filters, and aggregation all working together.

**RESULT ON OUR SAMPLE DATA**

| name | order_date | total |
|---|---|---|
| TechCorp | 2024-01-10 | 151,500 |
| TechCorp | 2024-04-05 | 5,000 |
| GlobalSolutions | 2024-01-20 | 25,000 |
| InnovationLabs | 2024-02-01 | 99,000 |
| AlphaInc | 2024-02-10 | 50,000 |
| GammaTech | 2024-03-01 | 50,000 |
| DeltaCorp | 2024-03-05 | 150,000 |
| EpsilonSoft | 2024-03-10 | 24,000 |
| OmegaLabs | 2024-04-01 | 75,000 |

9 rows returned.

## Q107. Self join for same city employees MEDIUM

```
SELECT e1.name AS emp1, e2.name AS emp2, e1.city
FROM employees e1
JOIN employees e2 ON e1.city = e2.city AND e1.emp_id < e2.emp_id;
```

**WHAT THIS QUERY DOES**

Another self join (see Q80), but this time joining the table to itself to find *pairs* of employees who share something in common — here, living in the same city. The condition `e1.emp_id < e2.emp_id` is a deliberate trick to avoid two kinds of redundancy: without it, you'd get both ('Aarav','Rahul') and ('Rahul','Aarav') as separate rows (the same pair, twice), and you'd also incorrectly match every employee with themself. Requiring the first ID to be strictly less than the second guarantees each pair appears exactly once.

**RESULT ON OUR SAMPLE DATA**

| emp1 | emp2 | city |
|---|---|---|
| Aarav Sharma | Ananya Menon | Mumbai |
| Aarav Sharma | Arjun Reddy | Mumbai |
| Aarav Sharma | Rahul Singh | Mumbai |
| Aarav Sharma | Ravi Teja | Mumbai |
| Priya Patel | Meera Krishnan | Delhi |
| Priya Patel | Vikram Reddy | Delhi |
| Rahul Singh | Ananya Menon | Mumbai |
| Rahul Singh | Arjun Reddy | Mumbai |
| Rahul Singh | Ravi Teja | Mumbai |
| Neha Gupta | Karthik Nair | Bangalore |
| Neha Gupta | Sneha Patel | Bangalore |
| Vikram Reddy | Meera Krishnan | Delhi |
| Ananya Menon | Arjun Reddy | Mumbai |
| Ananya Menon | Ravi Teja | Mumbai |
| Karthik Nair | Sneha Patel | Bangalore |
| Arjun Reddy | Ravi Teja | Mumbai |
| Rajesh Kumar | Suresh Babu | Chennai |
| Deepa Iyer | Kavya Nair | Hyderabad |

18 rows returned.

## Q108. Employees with managers in same department MEDIUM

```
SELECT e.name AS employee, m.name AS manager, e.department
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.department = m.department;
```

**WHAT THIS QUERY DOES**

Builds on the employee-manager self join from Q80, adding a filter for cases where the employee and their manager happen to be recorded in the exact same department — useful for spotting normal, expected reporting-line structures (versus, say, a cross-department dotted-line manager, which this filter would exclude).

**RESULT ON OUR SAMPLE DATA**

| employee | manager | department |
|---|---|---|
| Neha Gupta | Aarav Sharma | IT |
| Vikram Reddy | Aarav Sharma | IT |
| Ananya Menon | Priya Patel | HR |
| Karthik Nair | Aarav Sharma | IT |
| Arjun Reddy | Vikram Reddy | IT |
| Sneha Patel | Meera Krishnan | Marketing |
| Rajesh Kumar | Rahul Singh | Finance |
| Deepa Iyer | Priya Patel | HR |
| Suresh Babu | Vikram Reddy | IT |
| Kavya Nair | Rahul Singh | Finance |
| Ravi Teja | Aarav Sharma | IT |

11 rows returned.

## Q109. Cross join for department-location combinations HARD

```
SELECT d.dept_name, l.location
FROM departments d
CROSS JOIN (SELECT DISTINCT city AS location FROM employees) l;
```

**WHAT THIS QUERY DOES**

A `CROSS JOIN` is different from every other join in this level: it doesn't match on any condition at all — it simply pairs *every* row from the first table with *every* row from the second, producing all possible combinations (5 departments × 5 distinct cities = 25 rows here). This is useful for generating a complete grid of possibilities (e.g.

'which department/city combinations exist, and which don't') that you might later compare against actual data to spot gaps.

**RESULT ON OUR SAMPLE DATA**

| dept_name | location |
|---|---|
| Finance | Mumbai |
| Finance | Delhi |
| Finance | Bangalore |
| Finance | Chennai |
| Finance | Hyderabad |
| HR | Mumbai |
| HR | Delhi |
| HR | Bangalore |
| HR | Chennai |
| HR | Hyderabad |
| IT | Mumbai |
| IT | Delhi |
| IT | Bangalore |
| IT | Chennai |
| IT | Hyderabad |
| Marketing | Mumbai |
| Marketing | Delhi |
| Marketing | Bangalore |
| Marketing | Chennai |
| Marketing | Hyderabad |

Showing first 20 of 25 rows.

## Q110. Complex join with subquery HARD

```
SELECT e.name, e.salary, d.dept_name
FROM employees e
JOIN departments d ON e.department = d.dept_name
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department =
e.department);
```

**WHAT THIS QUERY DOES**

Combines a plain JOIN (to pull in the department's location details via d) with a **correlated subquery** in the WHERE clause (the same 'above your own department's average' check from Q123) — showing that JOINs and correlated

subqueries aren't an either/or choice; real queries frequently use both together, each solving a different part of the problem.

**RESULT ON OUR SAMPLE DATA**

| name | salary | dept_name |
|---|---|---|
| Priya Patel | 65,000 | HR |
| Rahul Singh | 85,000 | Finance |
| Vikram Reddy | 90,000 | IT |
| Karthik Nair | 95,000 | IT |
| Meera Krishnan | 68,000 | Marketing |
| Deepa Iyer | 64,000 | HR |
| Suresh Babu | 88,000 | IT |
| Ravi Teja | 91,000 | IT |

8 rows returned.